Übungsblatt 6

AKI/NES Programmierung 1, Wintersemester 2024/2025

Deadline: 27.11.2024, 23:00 Uhr

21.11.2024

Gehen Sie beim Lösen aller Programmieraufgaben strikt nach unserem Schema vor:

1. Schreiben Sie zuerst eine Kurzbeschreibung als Kommentar.
2. Führen Sie dann eine Datenanalyse durch. Schreiben Sie das Ergebnis Ihrer Datenanalyse ebenfalls als Kommentar in die Zeile nach der Kurzbeschreibung.
3. Schreiben Sie dann den Funktionskopf inkl. der Typen für Argumente und Ergebnis hin.
4. Schreiben Sie dann Tests mittels `check`.
5. Setzen Sie die passenden Schablonen in den Rumpf der Funktion ein.
6. Schreiben Sie erst dann den Funktionsrumpf hin.

Eine Lösung, in der nicht ersichtlich ist, dass Sie gemäß des Schemas aggiert haben, erhält *keine* Punkte. Insbesondere erhält eine Lösung, bei der nur der Funktionsrumpf da steht, keine Punkte, auch wenn die Funktion korrekt ist.

Dahingegen erhalten Sie Teilpunkte wenn Sie dem Schema folgen, aber an einem gewissen Punkt nicht weiterkommen.

Wichtiger Hinweis: Sie dürfen für Listen auf diesem Blatt keine Operationen verwenden, die eine Liste in-place modifizieren! Dazu gehören unter anderem: `remove`, `insert`, `append`, `extend`, `pop`, `sort` und auch die Zuweisung an einem Index mittels `l[i] = ...` oder auch `l += ...`.

Aufgabe 1 (4 Punkte)

Schreiben Sie eine Funktion `power`(x, n), welche die Potenz x^n berechnet. Sie können dabei annahmen dass n eine ganze Zahl ist. Benutzen Sie dazu eine Schleife. Eingebaute Methoden, Funktionen oder Operatoren, welche die Potenz berechnen, sind nicht erlaubt.

Aufgabe 2 (6 Punkte)

Schreiben Sie eine Funktion `reverse`, die eine Liste umdreht. Die Liste soll dabei *nicht* verändert werden. Eingebaute Methoden, Funktionen oder Operatoren wie Slicing, welche eine Liste umdrehen, sind nicht erlaubt.

Aufgabe 3 ((4+6 Punkte)

Es kommt häufig vor, dass Datensätzen unter einem bestimmten Schlüssel gespeichert werden und auch über diesen Schlüssel nachgeschlagen werden können. So sind die Daten von Studierenden über die Matrikelnummer zugreifbar, die Daten eines Kundens über die Kundennummer usw.

Eine einfache Repräsentation solcher Datensätzen ist eine Liste von Schlüssel-Wert-Paaren. Die Liste besteht dabei aus Paaren `[(`k_1`,`v_1`),(`k_2`,`v_2`), ...,(`k_n`,`v_n`)]`, wobei k_i ein Schlüssel ist (z.B. eine Matrikelnummer) und v_i ist der zugehörige Datensatz (z.B. die Stammdaten eines Studenten oder einer Studentin).

(a) Schreiben Sie eine Funktion `lookup`, die einen Schlüssel und eine Liste aus Schlüssel-Wert-Paaren akzeptiert. Rückgabewert soll der Wert zu dem Schlüssel sein. Falls der Schlüssel nicht in der Liste enthalten ist, soll `None` zurückgegeben werden.

(b) Schreiben Sie eine Funktion `insert`, die einen Schlüssel, einen Wert und einen solche Liste als Argumente nimmt. `insert` fügt das neue Schlüssel-Wert-Paar ein, falls jedoch schon ein Wert unter dem Schlüssel gespeichert ist, wird das alte Schlüssel-Wert-Paar vorher entfernt. Die ursprüngliche Liste soll dabei nicht verändert werden.

Beispiel: eine Person wird über eine Identifikationsnummer (eine Zahl) eindeutig identifiziert. In einer Liste aus Schlüssel-Wert-Paaren sollen nun unter der Identifikationsnummer die Namen der Personen gespeichert werden.

```
personList = [(42, 'Max'), (100, 'Helene'), (1, 'Verena'), (10, 'Dirk')]
```

- `lookup(10, personList)` liefert `'Dirk'`
- `lookup(11, personList)` liefert `None`
- `insert(101, 'Helene', personList)` liefert
  `[(42, 'Max'), (100, 'Helene'), (1, 'Verena'), (10, 'Dirk'), (101, 'Helene')]`
- `insert(100, 'Gundula, personList)` liefert
  `[(42, 'Max'), (100, 'Gundula'), (1, 'Verena'), (10, 'Dirk')]`

Die Reihenfolge der Liste, die von `insert` zurückgeliefert wird, ist nicht relevant, da jeder Schlüssel höchstens einmal vorkommen darf.

Aufgabe 4 (12 Punkte)

Matrizen sind in der Mathematik und in der Informatik allgegenwärtig. Eine $m \times n$ Matrix hat dabei m Zeilen und n Spalten mit Zahlen aus $\mathbb{R}$.

Wir können zwei Matrizen A und B multiplizieren, wenn A so viele Spalten wie B Zeilen hat. Sei also A ein $m \times n$ Matrix, also $A \in \mathbb{R}^{m \times n}$, und B eine $n \times p$ Matrix, also $B \in \mathbb{R}^{n \times p}$. Dann ist $A \cdot B = C \in \mathbb{R}^{m \times p}$ eine Matrix mit m Zeilen und p Spalten.

Beispiele:

$$D = \begin{pmatrix} 3 & 2 & 1 \\ 1 & 0 & 2 \end{pmatrix} \in \mathbb{R}^{2 \times 3}$$

$$E = \begin{pmatrix} 1 & 2 \\ 0 & 1 \\ 4 & 0 \end{pmatrix} \in \mathbb{R}^{3\times 2}$$

Wir schreiben für eine Matrix C den Eintrag in Zeile i und Spalte j als $c_{i,j}$. In den obigen Beispielen gilt also etwa $d_{1,2} = 2$ (in Matrix D) oder $e_{3,1} = 4$ (in Matrix E). Beachten Sie dass, anders als in Programmiersprachen üblich, die Nummerierung der Zeilen und Spalten bei 1 beginnt.

Das Produkt der zwei Matrizen $A \cdot B = C$ ist dann definiert als

$$c_{i,j} = \sum_{k=1}^{n} a_{i,k} \cdot b_{k,j}$$

für alle $1 \leq i \leq m$ und $1 \leq j \leq p$. Der Eintrag an i-ter Zeile und j-ter Spalte in der Ergebnismatrix ergibt sich also durch Multiplikation des i-ten Zeilenvektors von A mit dem j-ten Spaltenvektor von B.

Hier das Ergebnis der Multiplikation der beiden obigen Matrizen D und E:

$$\begin{pmatrix} 3 & 2 & 1 \\ 1 & 0 & 2 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 \\ 0 & 1 \\ 4 & 0 \end{pmatrix} = \begin{pmatrix} 3\cdot 1 + 2\cdot 0 + 1\cdot 4 & * \\ * & * \end{pmatrix} = \begin{pmatrix} 7 & * \\ * & * \end{pmatrix}$$

$$\begin{pmatrix} 3 & 2 & 1 \\ 1 & 0 & 2 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 \\ 0 & 1 \\ 4 & 0 \end{pmatrix} = \begin{pmatrix} 7 & 3\cdot 2 + 2\cdot 1 + 1\cdot 0 \\ * & * \end{pmatrix} = \begin{pmatrix} 7 & 8 \\ * & * \end{pmatrix}$$

$$\begin{pmatrix} 3 & 2 & 1 \\ 1 & 0 & 2 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 \\ 0 & 1 \\ 4 & 0 \end{pmatrix} = \begin{pmatrix} 7 & 8 \\ 1\cdot 1 + 0\cdot 0 + 2\cdot 4 & * \end{pmatrix} = \begin{pmatrix} 7 & 8 \\ 9 & * \end{pmatrix}$$

$$\begin{pmatrix} 3 & 2 & 1 \\ 1 & 0 & 2 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 \\ 0 & 1 \\ 4 & 0 \end{pmatrix} = \begin{pmatrix} 7 & 8 \\ 9 & 1\cdot 2 + 0\cdot 1 + 2\cdot 0 \end{pmatrix} = \begin{pmatrix} 7 & 8 \\ 9 & 2 \end{pmatrix}$$

Wir repräsentieren Matrizen in Python als verschachtelte Listen. Die äußere Liste repräsentiert dabei die Zeilen, die inneren Listen die Spalteneinträge der jeweiligen Zeile.

Beispielrepräsentation der beiden obigen Matrizen D und E in Python:

```
matrixD = [[3,2,1], [1,0,2]]
matrixE = [[1,2], [0,1], [4,0]]
```

Schreiben Sie eine Funktion `matrixMult`, die zwei Matrizen multipliziert.

Gesamtpunkte: 32

Abgabe

Via Upload im Praktomat bis **27.11.2024, 23:00 Uhr**. Speichern Sie jede Programmieren-aufgabe in eine eigene Datei. Der Name der Datei muss dabei `aufgabe_0`N`.py` sein, wobei N die Nummer der Aufgabe ist. Zum Beispiel lautet für Aufgabe 1 der Name der Datei dann `aufgabe_01.py`.

Verpacken Sie alle Dateien in einem zip-Archiv (Standard-Zip, keine 7zip, rar o.ä.), welches den Namen `NAME_MATRIKELNUMMER.zip` hat. Ersetzen Sie dabei `NAME` und `MATRIKELNUMMER` durch Ihren Namen und Ihre Matrikelnummer. Es ist wichtig, dass Sie sich an die Namenskonventionen halten, die wir Abgaben automatisch überprüfen.

Wichtig: Die Lösungen für **alle** Programmieraufgaben müssen ohne Fehler ausführbar sein, sonst akzeptiert der Praktomat Ihre Abgabe nicht. Wenn der Praktomat nach dem Upload für eine oder mehrere Aufgaben den Status **failed** anzeigt, wird ihre **komplette Abgabe nicht akzpetiert** und Sie erhalten für das **komplette Blatt 0 Punkte!** Kommentieren Sie fehlerhafte Codestellen in einem solche Fall aus, so dass der Praktomat Ihre Abgabe akzeptiert.